

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester IV)

AI(Artificial Intelligence)

Practical VI

Roll No.:T052	Name:Divyanjali yadav
Class:TYIT	Batch:01
Date of Assignment:01-08-2026	Date/Time of Submission:01-08-2026

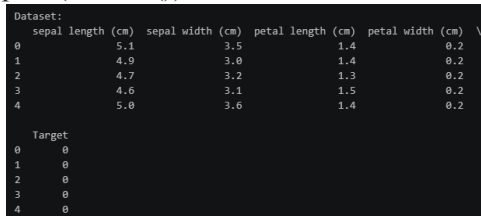
Machine Learning

1. Using any small dataset (e.g., Iris or a CSV dataset), write a Python program to demonstrate the basic machine learning workflow.

a. Load dataset using Pandas.

Code:

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
# a. Load Dataset using Pandas
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df["Target"] = iris.target
print("Dataset:")
print(df.head())
```



```
Dataset:
  sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1                3.5                1.4                0.2
1                4.9                3.0                1.4                0.2
2                4.7                3.2                1.3                0.2
3                4.6                3.1                1.5                0.2
4                5.0                3.6                1.4                0.2

  Target
0      0
1      1
2      2
3      3
4      4
```

b. Perform basic preprocessing (handling missing values or scaling).

b. Basic Preprocessing (Check Missing Values)

```
print("\nMissing Values:")
print(df.isnull().sum())
# Handle Missing Values (if any)
imputer = SimpleImputer(strategy="mean")
df[df.columns[:-1]] = imputer.fit_transform(df[df.columns[:-1]])
# Separate Features and Target
X = df.drop("Target", axis=1)
y = df["Target"]
```

Feature Scaling

```
scaler = StandardScaler()
X = scaler.fit_transform(X)
```

```
Missing Values:
sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
Target               0
dtype: int64
```

c. Split the dataset into training and testing sets.

```
# c. Split Dataset into Training and Testing Sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
print("\nTraining Samples:", len(X_train))
print("Testing Samples :", len(X_test))
```

```
Training Samples: 120
Testing Samples : 30
```

d. Train a simple classifier.

```
# d. Train a Simple Classifier
classifier = DecisionTreeClassifier(random_state=42)
classifier.fit(X_train, y_train)
```

```
DecisionTreeClassifier
Parameters
Fitted attributes
```

e. Display training and testing accuracy.

```
# e. Display Training and Testing Accuracy
# Predictions
train_pred = classifier.predict(X_train)
test_pred = classifier.predict(X_test)
# Accuracy
train_accuracy = accuracy_score(y_train, train_pred)
test_accuracy = accuracy_score(y_test, test_pred)
print("\nTraining Accuracy :", round(train_accuracy * 100, 2), "%")
print("Testing Accuracy :", round(test_accuracy * 100, 2), "%")
```

```
Training Accuracy : 100.0 %
Testing Accuracy : 100.0 %
```

world_cup_results.csv

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.impute import SimpleImputer
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
# a. Load Dataset using Pandas
df = pd.read_csv("world_cup_results.csv")
print("Dataset:")
print(df.head())
```

Dataset:

	Year	Date	Time	Round	Stadium	City	Country	\
0	1930	13-Jul-30	15:00	Group 1	Pocitos	Montevideo	Uruguay	
1	1930	13-Jul-30	15:00	Group 4	Parque Central	Montevideo	Uruguay	
2	1930	14-Jul-30	12:45	Group 2	Parque Central	Montevideo	Uruguay	
3	1930	14-Jul-30	14:50	Group 3	Pocitos	Montevideo	Uruguay	
4	1930	15-Jul-30	16:00	Group 1	Parque Central	Montevideo	Uruguay	

	HomeTeam	HomeGoals	AwayGoals	AwayTeam	Observation
0	France	4	1	Mexico	
1	USA	3	0	Belgium	
2	Yugoslavia	2	1	Brazil	
3	Romania	3	1	Peru	
4	Argentina	1	0	France	

b. Basic Preprocessing (Handling Missing Values)

```
print("\nMissing Values:")
print(df.isnull().sum())
```

```
Missing Values:
Year      0
Date      0
Time      0
Round     0
Stadium   0
City      0
Country   0
HomeTeam  0
HomeGoals 0
AwayGoals 0
AwayTeam  0
Observation 0
dtype: int64
```

Handle Missing Values

```
num_cols = df.select_dtypes(include=['int64', 'float64']).columns
cat_cols = df.select_dtypes(include=['object']).columns
```

Numerical columns

```
num_imputer = SimpleImputer(strategy="mean")
df[num_cols] = num_imputer.fit_transform(df[num_cols])
for col in cat_cols:
```

```
    df[col] = encoder.fit_transform(df[col])
```

Feature Scaling

```
scaler = StandardScaler()
X = scaler.fit_transform(X)
```

Separate Features and Target

```
X = df.drop("HomeTeam", axis=1)    # Features
y = df["HomeTeam"]
```

Categorical columns

```
cat_imputer = SimpleImputer(strategy="most_frequent")
df[cat_cols] = cat_imputer.fit_transform(df[cat_cols])
```

Encode Categorical Data

```
encoder = LabelEncoder()
```

c. Split Dataset into Training and Testing Sets

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
```

```

    test_size=0.2,
    random_state=42
)
print("\nTraining Samples:", len(X_train))
print("Testing Samples :", len(X_test))

```

```

Training Samples: 681
Testing Samples : 171

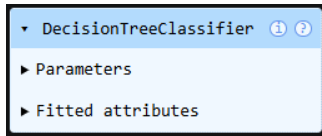
```

d. Train a Simple Classifier

```

classifier = DecisionTreeClassifier(random_state=42)
classifier.fit(X_train, y_train)

```



e. Display Training and Testing Accuracy

Predictions

```

train_pred = classifier.predict(X_train)
test_pred = classifier.predict(X_test)

```

Accuracy

```

train_accuracy = accuracy_score(y_train, train_pred)
test_accuracy = accuracy_score(y_test, test_pred)

```

```

print("\nTraining Accuracy :", round(train_accuracy * 100, 2), "%")
print("Testing Accuracy :", round(test_accuracy * 100, 2), "%")

```

```

Training Accuracy : 100.0 %
Testing Accuracy : 15.2 %

```

Comparison of Iris Dataset and World Cup Results Dataset

Feature	Iris Dataset	World Cup Results Dataset
Size	150 rows, 5 columns	852 rows, 12 columns
Data Type	Mostly numerical	Numerical + categorical
Target	Already available (Species)	Needs to be selected (e.g., HomeTeam)
Preprocessing	Simple	More complex
Use	Flower classification	Match/team prediction

Justification

- 1) Iris Dataset is small, clean, and easy to use. It is best for learning machine learning and classification algorithms.
- 2) World Cup Results Dataset is larger and contains real-world sports data. It requires more preprocessing (handling categorical data and feature selection) and is suitable for advanced analysis and prediction.

Conclusion:

The Iris dataset is better for beginners, while the World Cup Results dataset is better for real-world machine learning applications.